

KV Cache Engine

Interface Specification

LONGHORN SILICON LLM INFERENCE ACCELERATOR
Block 2 of 4 — KV Cache Compression / Decompression

Version: kv-isa-0.1 (first public draft)
Date: 2026-05-14
Status: Pre-tape-out stub for compiler integration
Author: Chaithu Talasila, The University of Texas at Austin
Reference: LonghornSilicon/kv-cache-engine

Scope. This document describes the externally-visible interface to the `kv_cache_engine` block as it will appear on the chip, on an FPGA prototype, or in the bit-accurate software model. A compiler backend targeting the KV cache engine writes against this interface; the hardware implementation must conform to it. This specification is stable for the KV cache engine block only and will be unified with the rest of the LonghornSilicon ISA when the Token Importance Unit and Memory Hierarchy Controller blocks land.

Contents

1	Block Overview	3
2	Address Space and Memory Map	3
3	Streaming Data Interfaces	3
3.1	s_axis_kv — Vector Write (Slave)	3
3.2	m_axis_kv — Vector Read (Master)	4
3.3	Eviction Interface	5
4	Logical Operations	5
4.1	Worked Lowering Example	5
4.2	Compiler Binding Patterns	6
5	C API Stub	6
6	Compression Algorithm: TurboQuant+	7
6.1	PolarQuant (Both K and V)	8
6.2	QJL Correction (K Path Only)	8
6.3	Bit Layout	8
7	Synthesis-Time Configuration	8
8	Bit-Accurate Reference	9
9	Integration Phases	9
10	Interaction with Other Blocks	9
11	Open Questions	9
12	Change Log	10

1. Block Overview

The KV cache engine manages on-chip storage of key and value tensors for transformer attention, with lossy compression to reduce DRAM bandwidth. It implements TurboQuant+ (`turbo4` configuration):

- **Key path:** PolarQuant (3-bit) + QJL (1-bit) = 4.25 bits per value (bpv)
- **Value path:** PolarQuant (3-bit) only ≈ 3.0 bpv

The compression pipeline consists of:

1. **Norm extraction:** L2 norm of the input vector, stored as metadata.
2. **Rotation:** Walsh-Hadamard Transform with random sign flips (deterministic from seed). Self-inverse, $O(n \log n)$ butterfly structure.
3. **Quantization:** 3-bit optimal Lloyd-Max scalar quantization on the rotated coordinates, producing centroid indices.
4. **QJL projection** (K path only): Quantized Johnson-Lindenstrauss transform on the residual. Random ± 1 projection matrix from seed, producing 1-bit signs per dimension.
5. **Packing:** Bit-packing of norm, indices, residual norm, and signs into a compact SRAM word.

Decompression reverses the pipeline: unpack, centroid lookup, QJL correction (K only), inverse WHT, norm rescaling.

Compression ratio: K = $3.56\times$, V = $4.92\times$, combined $\geq 3\times$.

Footprint: ~ 6400 FFs estimated (full pipeline); ~ 93 FFs current top-level.

Bit-exact reference: `sw/reference_model/kv_cache_engine_ref.py` and `sw/reference_model/kv_cache_engine_ref.{hpp,cpp}`.

2. Address Space and Memory Map

The block exposes a 256-byte AXI-Lite slave register window. All registers are 32-bit, word-aligned. The base address is set at chip integration time.

Conventions. RW = read/write; R = read-only; W1C = write-1-to-clear; (syn) = value fixed at synthesis time. Writes to read-only registers are silently dropped. Reading reserved offsets returns 0xDEADBEEF.

3. Streaming Data Interfaces

Two AXI-Stream interfaces carry the vector data. The AXI-Lite window in §2 is for control only.

3.1 `s_axis_kv` — Vector Write (Slave)

Protocol:

- A complete vector is exactly `VECTOR_DIM` beats. `tlast` asserts on the final beat.

Table 1: AXI-Lite register window (offsets relative to base address).

Offset	Name	Access	Reset	Purpose
0x00	CTRL	RW	0x0	bit[0]: <code>soft_reset</code> (write-1 to pulse); bit[1]: <code>enable</code>
0x04	STATUS	R	0x1	bit[0]: <code>idle</code> ; bit[1]: reserved; bit[2]: re- served; bit[3]: <code>sram_full</code>
0x08	INFO_DIM	R	(syn)	Synthesis-time <code>VECTOR_DIM</code>
0x0C	INFO_PQ_BITS	R	(syn)	PolarQuant quantization bits (3)
0x10	INFO_QJL_BITS	R	(syn)	QJL projection bits (1)
0x14	INFO_SRAM_DEPTH	R	(syn)	Number of SRAM entries
0x18	INFO_CR_K	R	(syn)	Key compression ratio, fixed-point 8.8
0x1C	INFO_CR_V	R	(syn)	Value compression ratio, fixed-point 8.8
0x20	INFO_VERSION	R	0x00010000	bits[31:24] major, bits[23:16] minor, bits[15:8] patch, bits[7:0] build
0x24	OCCUPANCY	R	0x0	Number of valid entries in SRAM
0x28	WRITE_ADDR	RW	0x0	Target SRAM address for next write
0x2C	READ_ADDR	RW	0x0	Target SRAM address for next read
0x30	KV_SELECT	RW	0x0	bit[0]: 0 = key path, 1 = value path
0x34	IRQ_MASK	RW	0x0	bits[3:0]: interrupt enable mask
0x38	IRQ_STATUS	R/W1C	0x0	bits[3:0]: interrupt pending; write-1-to- clear

Signal	Width	Direction	Purpose
<code>tdata</code>	<code>COORD_WIDTH</code>	in	One signed two's-complement coordinate
<code>tlast</code>	1	in	Assert on the final coordinate of the vector
<code>tvalid</code>	1	in	Handshake: data is valid
<code>tready</code>	1	out	Handshake: block is ready to accept
<code>tuser</code>	1	in	0 = key vector, 1 = value vector

- The `tuser` field selects the compression path: key vectors receive PolarQuant + QJL compression; value vectors receive PolarQuant only.
- The target SRAM address must be set via `WRITE_ADDR` before streaming begins.
- Backpressure: when the block is not ready (e.g., during compression or SRAM write), `tready` deasserts. Upstream must hold signals stable.

3.2 `m_axis_kv` — Vector Read (Master)

Signal	Width	Direction	Purpose
<code>tdata</code>	<code>COORD_WIDTH</code>	out	One signed two's-complement decompressed coordi- nate
<code>tlast</code>	1	out	Asserted on the final coordinate
<code>tvalid</code>	1	out	Handshake: data is valid
<code>tready</code>	1	in	Handshake: downstream is ready to accept

Protocol. One beat per coordinate, `VECTOR_DIM` beats per vector. Set `READ_ADDR` and `KV_SELECT` before initiating a read. The decompression path is selected by `KV_SELECT`.

3.3 Eviction Interface

Signal	Width	Direction	Purpose
<code>evict_needed</code>	1	out	Asserted when SRAM is full
<code>evict_addr</code>	$\lceil \log_2 D \rceil$	out	Address of entry to evict

This interface connects to the Memory Hierarchy Controller (block 4). When `evict_needed` asserts, the controller must either spill the entry at `evict_addr` to DRAM or invalidate it before the next write.

4. Logical Operations

Below are the abstract operations a compiler emits. Each maps to a short sequence of register writes and stream beats; the C API in §5 gives the concrete library calls.

Operation	Inputs	Outputs	Description
<code>KV_QUERY</code>	—	INFO struct	Read all <code>INFO_*</code> registers
<code>KV_RESET</code>	—	—	Soft reset: clear SRAM, reset FSM
<code>KV_ENABLE</code>	—	—	Set bit[1] of CTRL
<code>KV_COMPRESS_KEY</code>	<code>addr</code> , D coords	—	Compress and store a key vector at <code>addr</code>
<code>KV_COMPRESS_VALUE</code>	<code>addr</code> , D coords	—	Compress and store a value vector at <code>addr</code>
<code>KV_DECOMPRESS_KEY</code>	<code>addr</code>	D coords	Read and decompress a key vector from <code>addr</code>
<code>KV_DECOMPRESS_VALUE</code>	<code>addr</code>	D coords	Read and decompress a value vector from <code>addr</code>
<code>KV_STATUS</code>	—	STATUS bits	Read the STATUS register
<code>KV_OCCUPANCY</code>	—	count	Read the OCCUPANCY register

4.1 Worked Lowering Example

This section walks through how a compiler lowers a transformer attention operation into KV cache engine operations.

Suppose the compiler’s IR contains:

```
%out = transformer_block(%q, %k, %v, %kv_cache)
      : tensor<S x D x f16>, tensor<S x D x f16>, tensor<S x D x f16>,
        kv_cache_handle -> tensor<S x D x f16>
```

Stage 1: setup. Query hardware configuration once per compilation:

```
info = KV_QUERY()           // -> dim=64, pq_bits=3, qjl_bits=1, sram_depth
    =16
KV_RESET()
KV_ENABLE()
```

Stage 2: store phase. For each new token, compress and store its K/V:

```

for token_idx in new_tokens:
    addr = token_idx % info.sram_depth
    KV_COMPRESS_KEY(addr, k_vectors[token_idx])
    KV_COMPRESS_VALUE(addr, v_vectors[token_idx])

```

Stage 3: attention phase. Decompress cached K/V for attention:

```

for cached_idx in range(num_cached):
    k_hat = KV_DECOMPRESS_KEY(cached_idx)
    v_hat = KV_DECOMPRESS_VALUE(cached_idx)
    score = dot(query, k_hat) / sqrt(D)

    // Route through precision controller (block 1)
    decision = PC_PUSH_TILE(score)
    if decision == FP16:
        out += fp16_matmul(softmax(score), v_hat)
    else:
        out += int8_matmul(softmax(score), v_hat)

```

Note the interaction between block 1 (precision controller) and block 2 (KV cache engine): the decompressed K/V vectors feed into the attention score computation, which the precision controller routes.

4.2 Compiler Binding Patterns

MLIR dialect. Define `lhsi.kv.*` ops mirroring the table in §4. The lowering pass rewrites `linalg.attention` or vendor-equivalent ops to emit `lhsi.kv.compress_key`, `lhsi.kv.decompress_key`, etc.

TVM Relax / TIR. Register backend target "`lhsi-kv`", pattern-match KV cache operations in the attention sub-graph, emit calls to TVM-runtime extern functions named for each `KV_*` operation.

ONNX. Add `CustomOpDomain` with KV cache ops, graph transform inserts compression/decompression around attention.

Custom IR. Replace KV cache codegen with `KV_*` operation sequences targeting the C API.

5. C API Stub

```

/* lhsi_kv_cache_engine.h --- LonghornSilicon KV Cache Engine driver API
. */

#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

typedef struct {
    uint32_t vector_dim;
    uint32_t pq_bits;
    uint32_t qjl_bits;
    uint32_t sram_depth;

```

```

    uint32_t cr_k_fixed;          /* compression ratio K, fixed-point 8.8
        */
    uint32_t cr_v_fixed;          /* compression ratio V, fixed-point 8.8
        */
    uint32_t version;
} lhsi_kv_info_t;

typedef struct lhsi_kv_handle lhsi_kv_handle_t;

/* Open / close */
int lhsi_kv_open (const char *device, lhsi_kv_handle_t **out);
void lhsi_kv_close(lhsi_kv_handle_t *h);

/* Configuration queries */
int lhsi_kv_query(lhsi_kv_handle_t *h, lhsi_kv_info_t *out);

/* Control */
int lhsi_kv_reset (lhsi_kv_handle_t *h);
int lhsi_kv_enable(lhsi_kv_handle_t *h, bool enable);

/* Compress and store a key vector at the given SRAM address. */
int lhsi_kv_compress_key(lhsi_kv_handle_t *h, uint32_t addr,
                        const int16_t *coords, size_t dim);

/* Compress and store a value vector at the given SRAM address. */
int lhsi_kv_compress_value(lhsi_kv_handle_t *h, uint32_t addr,
                        const int16_t *coords, size_t dim);

/* Read and decompress a key vector from the given SRAM address. */
int lhsi_kv_decompress_key(lhsi_kv_handle_t *h, uint32_t addr,
                        int16_t *coords, size_t dim);

/* Read and decompress a value vector from the given SRAM address. */
int lhsi_kv_decompress_value(lhsi_kv_handle_t *h, uint32_t addr,
                        int16_t *coords, size_t dim);

/* Diagnostics. */
typedef struct {
    bool      idle;
    bool      sram_full;
    uint32_t  occupancy;
} lhsi_kv_status_t;
int lhsi_kv_status(lhsi_kv_handle_t *h, lhsi_kv_status_t *out);

```

Return codes follow the standard Linux convention: 0 on success, `-errno` on failure.

6. Compression Algorithm: TurboQuant+

TurboQuant+ is a two-stage vector quantization scheme designed for KV cache compression in transformer inference. The `turbo4` configuration provides 4.25 bpv for keys and ~ 3.0 bpv for values.

6.1 PolarQuant (Both K and V)

1. **Norm extraction:** Compute $\|x\|_2$ (L2 norm) and normalize: $\hat{x} = x/\|x\|_2$.
2. **Rotation:** Apply random sign flips $s_i \in \{-1, +1\}$ (from LCG seed), then Walsh-Hadamard Transform: $y = \text{WHT}(\hat{x} \odot s)$. This randomizes the coordinate distribution toward i.i.d. Gaussian.
3. **Quantization:** 3-bit optimal Lloyd-Max scalar quantization of each rotated coordinate y_i , using 8 centroids symmetric around zero: $\pm 0.2451\sigma, \pm 0.7560\sigma, \pm 1.3439\sigma, \pm 2.1520\sigma$ where $\sigma = 1/\sqrt{D}$.

6.2 QJL Correction (K Path Only)

After PolarQuant, the quantization residual $r = y - \hat{y}$ is projected through a random ± 1 matrix $R \in \{-1, +1\}^{D \times D}$ (from seed + 0xDEADBEEF):

$$z_j = \text{sign}\left(\sum_{i=1}^D R_{ji} \cdot r_i\right)$$

The 1-bit signs z_j preserve inner products in expectation (Johnson-Lindenstrauss property). During decompression, the QJL correction is added back to the dequantized PolarQuant output, improving attention score accuracy for keys.

6.3 Bit Layout

Path	Fields	Bits	Compression
Key	$\text{norm}(16) + \text{indices}(D \times 3) + \text{res_norm}(16) + \text{signs}(D \times 1)$	$32 + 4D$	$3.56\times$
Value	$\text{norm}(16) + \text{indices}(D \times 3)$	$16 + 3D$	$4.92\times$

For $D = 64$: key = 288 bits, value = 208 bits, vs. uncompressed $D \times 16 = 1024$ bits.

7. Synthesis-Time Configuration

Parameter	Default	Range	Notes
VECTOR_DIM	64	16, 32, 64, 128	Must be a power of two (WHT constraint)
PQ_BITS	3	2, 3, 4	Lloyd-Max quantization bits
QJL_BITS	1	1	Fixed at 1 in kv-isa-0.1
SRAM_DEPTH	16	4–4096	Behavioral reg array for Sky130; real macros at 16FFC
COORD_WIDTH	16	8, 12, 16	Fixed-point coordinate width
COORD_FRAC	12	4–14	Fractional bits in coordinate representation
NORM_WIDTH	16	8, 12, 16	Norm storage width
ROTATION_SEED	42	any 32-bit	Deterministic LCG seed for sign flips and QJL matrix

8. Bit-Accurate Reference

Three implementations are provided, all bit-exact with each other:

1. **Python:** `sw/reference_model/kv_cache_engine_ref.py` — golden reference with fixed-point arithmetic.
2. **C++:** `sw/reference_model/kv_cache_engine_ref.{hpp,cpp}` — same fixed-point math, with `extern "C"` API using opaque handles.
3. **SystemVerilog:** `rtl/kv_cache_engine.sv` + sub-modules — synthesizable RTL.

Usage:

```
from kv_cache_engine_ref import KVCacheEngine, KVCacheEngineInfo

engine = KVCacheEngine()
ck = engine.compress_key(vector)
reconstructed = engine.decompress_key(ck)
```

9. Integration Phases

Phase	Timeline	Compiler targets	We provide
0	now	Bit-accurate Python and C++ reference models	Models + ISA + tests
1	when FPGA arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + driver
2	after all 4 blocks complete	Multi-block project	FPGA Integrated attention pipeline
3	post-tape-out (2027+)	PCIe-attached chip	custom TSMC 16FFC silicon + Linux driver

10. Interaction with Other Blocks

- **Block 1 (Precision Controller):** Receives decompressed K/V vectors from this block for attention score computation. The precision controller decides INT8 vs FP16 routing based on the decompressed scores.
- **Block 3 (Token Importance Unit):** Provides token importance scores that inform eviction decisions when the SRAM is full.
- **Block 4 (Memory Hierarchy Controller):** Receives `evict_needed` / `evict_addr` signals when SRAM is full. Manages spilling compressed entries to DRAM and fetching them back.

11. Open Questions

1. Should the compression path support variable-rate quantization (2/3/4-bit PolarQuant selected per-vector based on norm)?

2. Should the QJL projection dimension be configurable (currently $D \times D$, could be $D \times D'$ with $D' < D$ for reduced storage)?
3. Should the block support direct SRAM-to-SRAM copy for cache migration?
4. Are the current fixed-point widths sufficient for larger models ($D = 128, 256$)?
5. Should the eviction policy be configurable via registers (LRU, FIFO, importance-based)?

12. Change Log

- kv-isa-0.1 (2026-05-14): First public draft. Stable for the KV cache engine block only.

LonghornSilicon KV Cache Engine — Interface Specification. This document is open and may be redistributed. The TSMC 16FFC PDK referenced in the integration phases is NDA-protected and is not part of this document or the public repository.